K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

TRUSTBOX

Hritunjaya Chauhan-2401420024
Jahanvi Deswal-2401420026
Kashish Sharma-2401420039

Under Supervison of
Internal : Dr. Appurva Jain, External:
School of Engineering and Technology

Roles of Each Student

Person	Role	Main Tasks
Hritunjaya	HTML Structure Developer	Create HTML layout, forms, sections, and labels
Jahanvi	JavaScript Developer	App logic, OTP, interactivity, localStorage
Kashish	CSS UI/UX Designer	Add all styles, effects, animations, responsive design

Project OVERVIEW

•TrustBox is a secure and verified anonymous reporting system designed for schools and colleges to help students report sensitive issues like bullying, harassment, or academic misconduct without fear. Students authenticate using their college email and receive a unique anonymous ID through a one-time password (OTP). This ensures genuine complaints while preserving student privacy. Repeated complaints from different IDs are prioritized, while suspicious patterns from the same ID are flagged — helping institutions take timely, focused action and reduce false reporting.

OBJECTIVES

1. Safe and Anonymous Reporting – Students should be allowed to make reports confidentially, knowing they will not be exposed.

2. Verified Entry via Confidential Unique ID – It provides each student a unique confidential ID for verification purposes so that no false complaints can be made while protecting their anonymity.

3. Efficient and Secure Complaint Handling – Protective data encryption, as well as the availability of an admin panel where authorized personnel will solve complaints responsibly.

PROBLEM STATEMENT

Real-World Problem TrustBox is Solving:

Many students in educational institutions face issues such as harassment, academic pressure, infrastructure problems, or administrative neglect — but hesitate to report them. The core reason is fear of retaliation, exposure, or not being taken seriously. As a result, genuine complaints often go unreported, leaving problems unresolved and affecting student well-being.

Why This Problem is Important & How TrustBox Helps:

This lack of safe reporting harms both students and the institution's reputation. Without feedback, schools cannot improve or protect students effectively.

TrustBox solves this by providing a secure, anonymous complaint submission system:

1.Students verify through OTP (to prove they're genuine).

2. Their identity remains hidden.

3.The system flags repeated complaints for priority action.

This encourages students to speak up safely, helping institutions to identify and solve critical issues early.

METHODOLOGY

The TrustBox system follows a structured development process to address anonymous student reporting through secure authentication and complaint handling. The workflow includes:

1- User Authentication via OTP

2- Anonymous Complaint Submission

3- Secure Data Handling and Storage

4.Admin Access for Complaint Review

5.Complaint Prioritization (based on frequency)

SYSTEM ARCHITECTURE

Layer	Technology Used	Description
Frontend	HTML, CSS, JavaScript	User interface for students and admins
OTP Engine	JavaScript (Simulated)	Sends and verifies OTP for user validation
Anonymous Layer	JavaScript Logic	Assigns unique Anonymous IDs after OTP verification
Storage	localStorage (or MongoDB in full version)	Temporarily stores complaints
Admin Panel	JavaScript + DOM	Allows viewing submitted complaints

FLOWCHART

[Student Enters Email]

[Send OTP Simulated]

[Enter OTP Verified]

[Anonymous ID Assigned]

[Write & Submit Complaint]

[Complaint Saved to Storage]

[Admin Views Complaints]

Functional Approach

A. OTP-Based Verification

User inputs college email.

OTP is generated on the frontend (simulated for demo).

OTP is verified; upon success, user receives a random anonymous ID.

B. Complaint Submission

Verified users can write and submit complaints.

Complaints are stored with an anonymous ID in localStorage (can be replaced with MongoDB later).

Each complaint includes:

Anonymous ID

Complaint Text

Timestamp (optional)

C. Admin Interface

Admin logs in using a predefined password.

All submitted complaints are retrieved and displayed.

Interface includes visual blocks for each complaint.

Can be extended to include:

Filtering by category

Marking status (Pending/In Progress/Resolved)

TOOLS AND TECHNOLOGIES USED

Technologies Used:

HTML:

Defines the structure and semantic layout (buttons, inputs, divs).

CSS:

Embedded in the <style> tag to style elements with gradients, animations, shadows, and rounded corners.

JavaScript :

Handles UI logic, OTP generation, form transitions, and local data storage using localStorage.

FRONT-END

Layout & Structure:

Single Page Interface:

Uses hidden sections like #userSection, #adminSection, etc., shown/hidden dynamically using JavaScript.

Buttons and Inputs:

Buttons trigger different flows (login, submit complaint, view complaints).

Input fields for OTP, email, password, and complaints.

Styling Details:

Dark Mode Theme:

Dominant colors: deep maroon, black, gray.

Visually calming, privacy-friendly tone fitting an anonymous system.

Neumorphism Elements:

Soft shadows and gradients create depth on input boxes and buttons.

Responsive Design:

Container width set with percentages and max-width.

Central alignment with Flexbox ensures adaptability across devices.

Functional Logic:

OTP Simulation:

Random 6-digit OTP is generated and shown via alert() (not sent via email).Admin Authentication:

Simple password check (admin123) for demo purposes.

Local Storage:

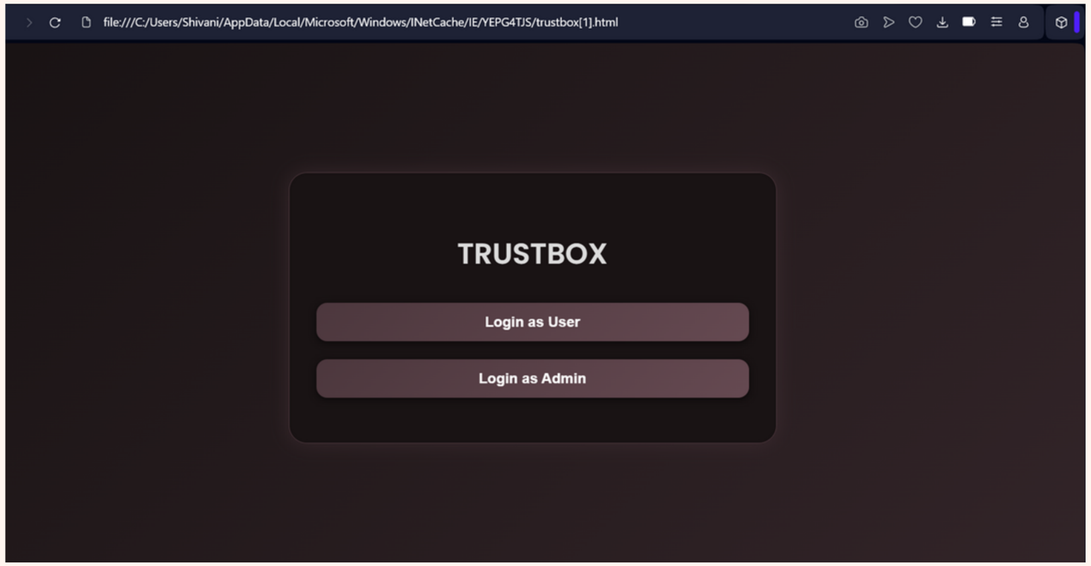
Complaints are saved and retrieved using localStorage.

UX/UI DESIGN

Wireframe-Like Structure:
Home View: Choose "User" or "Admin" login.
User View:
Enter email → send OTP
Enter OTP → login anonymously
Submit complaint
Admin View:
Enter password
View complaints with status tags
User Experience (UX) Principles Applied:
Simplicity: Clear flow for each role (User/Admin), minimal navigation.
Feedback:
Alerts for wrong OTP, invalid email, submission confirmation.
Hover animations provide tactile feedback on buttons.
Status Indicators:
Each complaint is marked as:
Normal
Priority (same complaint from multiple users)
Suspicious (same user submitted same complaint multiple times)
Tools :
While not used here, for actual wireframing and prototyping, designers might use:
Figma
Adobe XD
Sketch
Design Rationale:
Encourage anonymous, honest reporting by:
Not requiring passwords for users.
Providing feedback after submission.
Using a clean, non-intimidating interface.
Admin view focuses on pattern detection in complaints using JavaScript analysis.

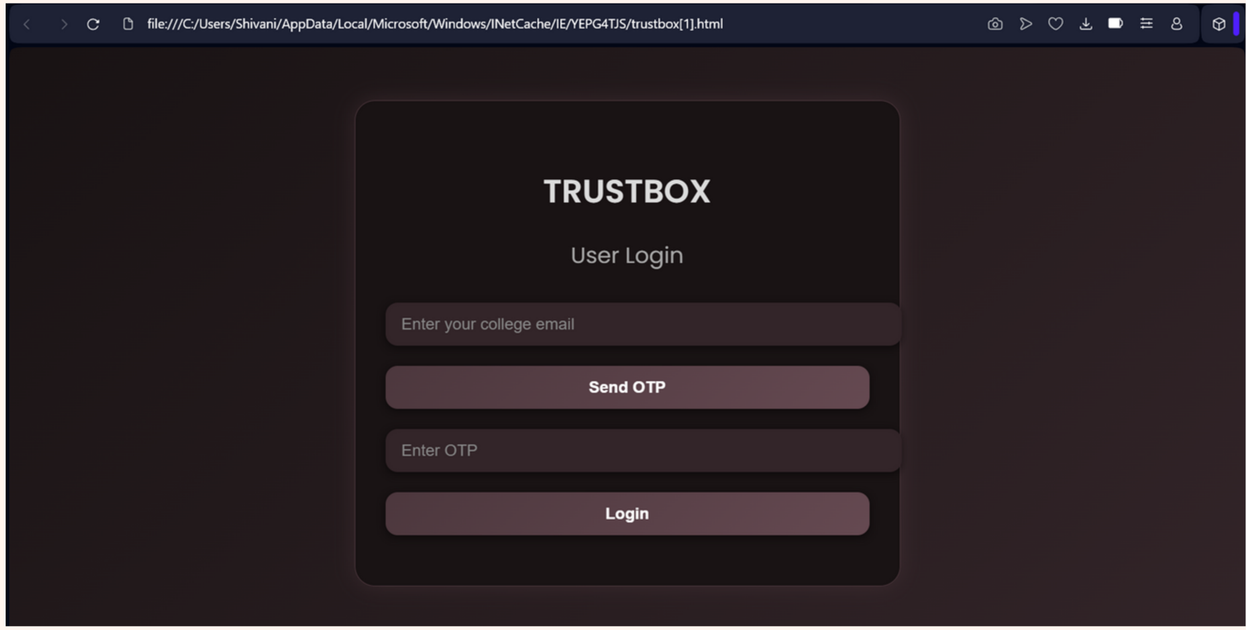
DEMONSTRATION

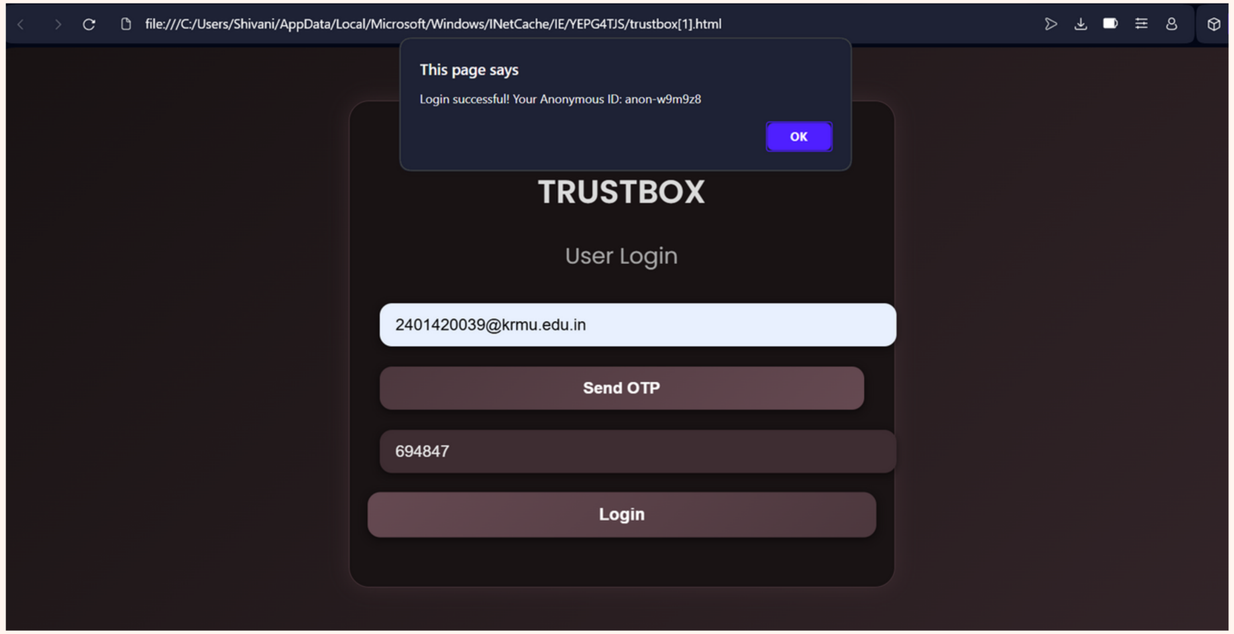
USER LOGIN



Step 1-Login as a user to register a complaint

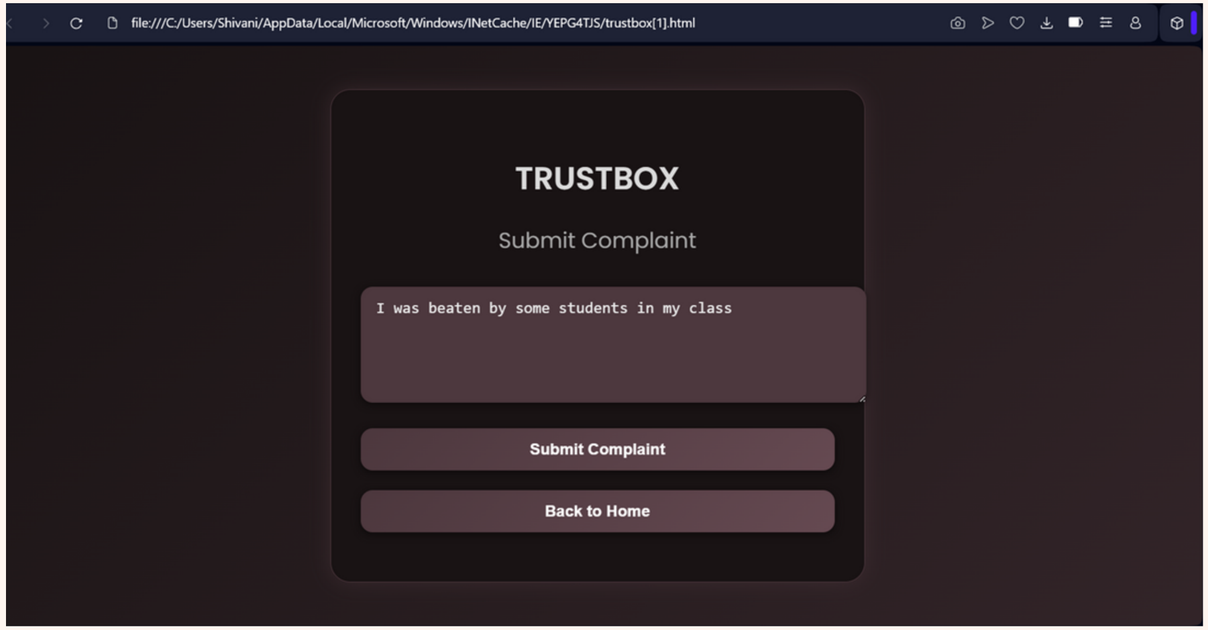
Step 2 - Enter your university mail id (@krmu.edu.in)
Click on send otp and an otp will be provided .
Click login



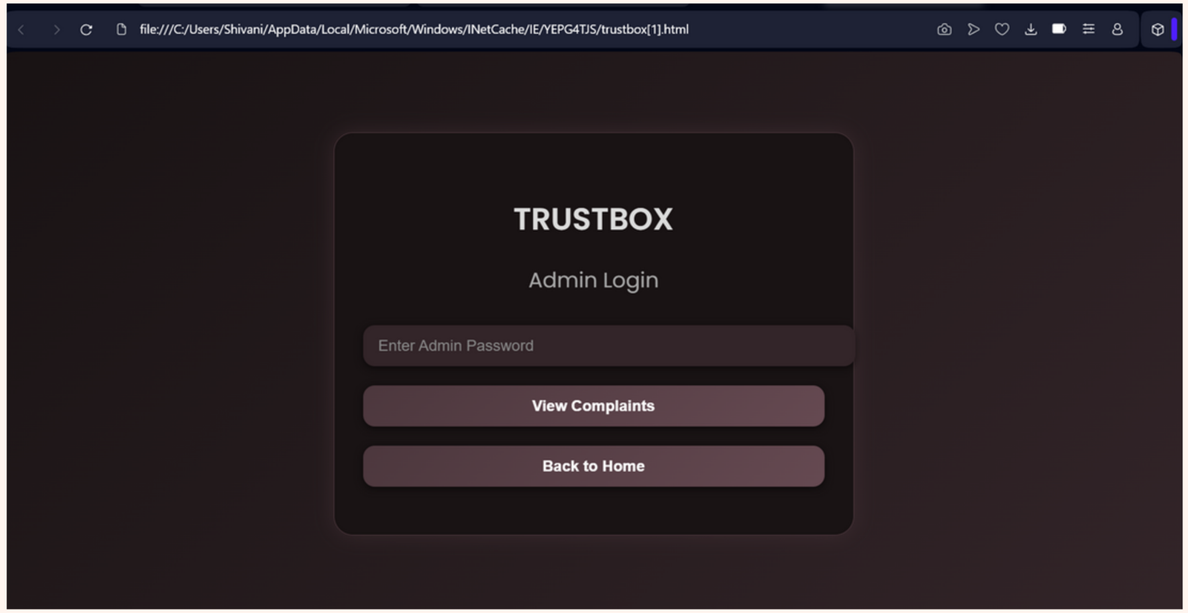


Step 3- An anonymous id will be assigned to the user

Step 4 - User will register their complaint .
once the complain is registered the user will be directed to the home page

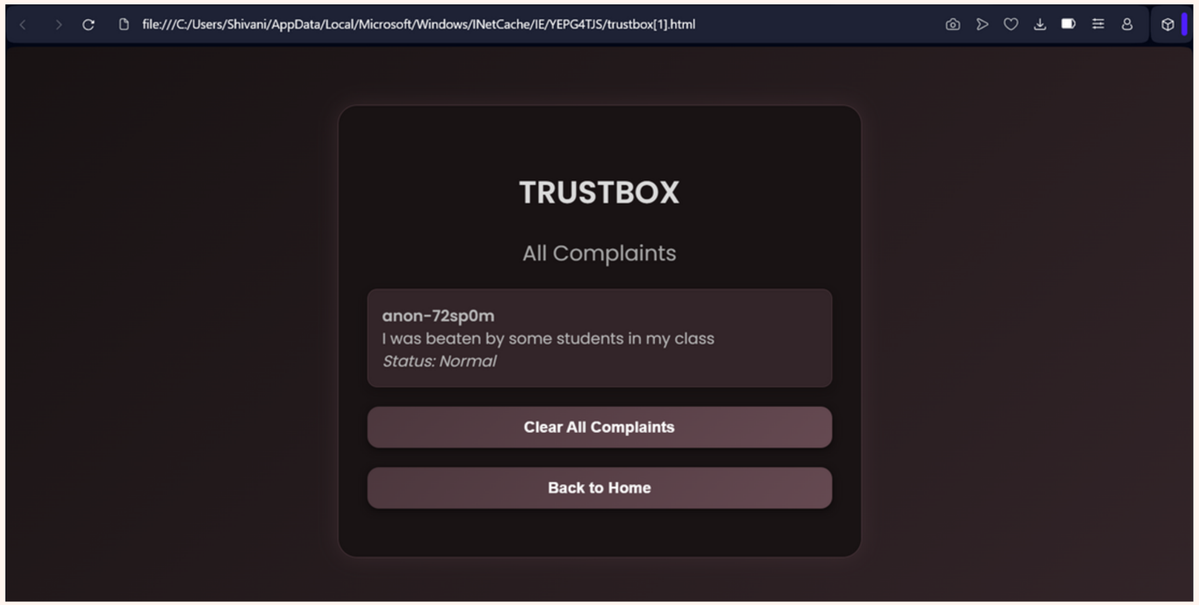


ADMIN LOGIN



Step 5 - Admin will enter their password to login and view student’s complaint

Step 6 - Complaints will be displayed .
Admin can also clear all the complaints.



Key Features

Anonymous login with college email (only @krmu.edu.in is allowed)

OTP-based verification (simulated)

Auto-generated anonymous IDs

Complaint submission form

Admin view with complaint analysis

Priority: Same complaint reported by multiple users

Suspicious: Same complaint reported multiple times by one user

Ability to clear all complaints (admin only)

Responsive UI with animations and secure interaction prompts

Learning from the project

Project Reflection

Learning and Experience:

Through this project, we gained valuable hands-on experience in building a complete web application from scratch. We learned how to:

Design a clean, responsive frontend using HTML, CSS, and animations to enhance user experience.

Implement client-side functionality with JavaScript, including email validation, OTP generation, anonymous user IDs, complaint submissions, and admin authentication.

Manage data storage using localStorage while understanding its limitations in terms of security and scalability.

Collaborate effectively by dividing roles — focusing on frontend design, application logic.— which gave us a real-world feel of teamwork in a software development project.

Identify security challenges and explore ways to make data handling more secure.

This project helped strengthen our skills in problem-solving, coding best practices, and user-centric design thinking

FUTURE ENHANCEMENTS

We have identified several areas where we can improve and extend the application:

Implement a Real Backend:
Move from localStorage to a secure backend database like Firebase or MongoDB with an Express.js server.

Email-based OTP Delivery:
Integrate a real email service (e.g., SendGrid, EmailJS) to send OTPs securely instead of simulating them.

Enhanced Admin Dashboard:
Create a full dashboard for the admin with filtering options, complaint categories, and analytics (such as number of complaints per week).

User Session Management:
Add session timeouts and secure login/logout functionality.

Mobile Responsiveness:
Improve UI for small devices and add offline capabilities using Service Workers.

Complaint Status Tracking:
Allow admins to update the status of complaints (Pending, In Review, Resolved).

Multi-Admin Support:
Enable multiple admin accounts with role-based access control.

Thank You